

Common Table Expressions (CTE) - MySQL

What is a CTE?

A Common Table Expression (CTE) is a temporary named result set created within a query using the WITH clause.

CTEs help break large queries into smaller readable blocks.

A CTE exists only during query execution.

Why Use CTE?

For a Java Backend Developer, queries often become complex because of:

- Multiple joins
- Aggregations
- Reporting logic
- Filtering on derived data
- Nested subqueries
- Reusing intermediate results

CTEs improve:

- Readability
- Maintainability
- Debugging
- Query organization

CTEs are commonly used in:

- Dashboard APIs
- Reporting APIs
- Analytics queries
- Data transformation logic
- Hierarchical queries

Problem Without CTE

Find employees earning more than their department average salary.

Without CTE:

```
SELECT e.*
FROM employee e
JOIN (
    SELECT department, AVG(salary) AS avg_salary
    FROM employee
    GROUP BY department
) dept_avg
ON e.department = dept_avg.department
WHERE e.salary > dept_avg.avg_salary;
```

Problem:

- Query becomes harder to read
- Logic is embedded inside JOIN
- Difficult to debug when query grows

Same Query Using CTE

```
WITH dept_avg AS (
    SELECT department, AVG(salary) AS avg_salary
    FROM employee
    GROUP BY department
)
SELECT e.*
FROM employee e
JOIN dept_avg d
ON e.department = d.department
WHERE e.salary > d.avg_salary;
```

Advantages:

- Step-by-step query structure & Better readability
- Easier maintenance & Cleaner debugging

CTE Syntax

```
WITH cte_name AS (  
    SELECT column1, column2  
    FROM table_name  
    WHERE condition  
)  
SELECT *  
FROM cte_name;
```

Lab Setup

Create Database

```
CREATE DATABASE cte_lab;  
USE cte_lab;
```

Create Table

```
CREATE TABLE employee (  
    emp_id INT PRIMARY KEY,  
    name VARCHAR(50),  
    department VARCHAR(50),  
    salary DOUBLE  
);
```

Insert Data

```
INSERT INTO employee VALUES  
(1, 'Surya', 'IT', 50000),  
(2, 'Rahul', 'HR', 60000),  
(3, 'Ankit', 'IT', 75000),  
(4, 'Neha', 'Finance', 40000),  
(5, 'Priya', 'IT', 90000),  
(6, 'Kiran', 'HR', 55000);
```

CTE Queries

1. Basic CTE

Get employees with salary greater than 60000.

```
WITH high_salary AS (  
    SELECT *  
    FROM employee  
    WHERE salary > 60000  
)  
SELECT *  
FROM high_salary;
```

Creates a temporary result set containing high salary employees.

2. CTE with Aggregation

Find department-wise average salary.

```
WITH dept_avg AS (  
    SELECT department, AVG(salary) AS avg_salary  
    FROM employee  
    GROUP BY department  
)  
SELECT *  
FROM dept_avg;
```

Stores aggregated department salary data in a temporary result.

3. Employees Above Average Salary

Find employees earning above overall average salary.

```
WITH avg_salary_cte AS (  
    SELECT AVG(salary) AS avg_salary  
    FROM employee  
)  
SELECT *  
FROM employee, avg_salary_cte  
WHERE employee.salary > avg_salary_cte.avg_salary;
```

Explanation:

Separates average salary calculation from filtering logic.

4. Employees Above Department Average

Find employees earning above their department average salary.

```
WITH dept_avg AS (  
    SELECT department, AVG(salary) AS avg_salary  
    FROM employee  
    GROUP BY department  
)  
SELECT e.*  
FROM employee e  
JOIN dept_avg d  
ON e.department = d.department  
WHERE e.salary > d.avg_salary;
```

First calculates department average salary, then compares employees against it.

5. Multiple CTEs

Compare IT average salary and HR average salary.

```
WITH it_avg AS (  
    SELECT AVG(salary) AS avg_salary  
    FROM employee  
    WHERE department = 'IT'  
),  
hr_avg AS (  
    SELECT AVG(salary) AS avg_salary  
    FROM employee  
    WHERE department = 'HR'  
)  
SELECT  
    it_avg.avg_salary AS it_salary,  
    hr_avg.avg_salary AS hr_salary  
FROM it_avg, hr_avg;
```

Explanation:

Multiple CTEs help separate different logical blocks.

6. CTE with JOIN

Find employees from departments having average salary greater than 60000.

```
WITH dept_avg AS (  
    SELECT department, AVG(salary) AS avg_salary  
    FROM employee  
    GROUP BY department  
)  
SELECT e.*  
FROM employee e  
JOIN dept_avg d  
ON e.department = d.department  
WHERE d.avg_salary > 60000;
```

CTE is used as a reusable derived table during JOIN.

7. CTE with Filtering

Get IT employees with salary greater than 70000.

```
WITH it_employees AS (  
    SELECT *  
    FROM employee  
    WHERE department = 'IT'  
)  
SELECT *  
FROM it_employees  
WHERE salary > 70000;
```

Filters are separated into stages for readability.

8. CTE Reuse Example

Reuse filtered employee data multiple times.

```
WITH high_salary AS (  
    SELECT *  
    FROM employee  
    WHERE salary > 60000  
)  
SELECT * FROM high_salary  
UNION ALL  
SELECT * FROM high_salary;
```

Same temporary result is reused multiple times.

9. Recursive CTE

Generate numbers from 1 to 5.

```
WITH RECURSIVE numbers AS (  
    SELECT 1 AS num  
  
    UNION ALL  
  
    SELECT num + 1  
    FROM numbers  
    WHERE num < 5  
)  
SELECT *  
FROM numbers;
```

Recursive CTE repeatedly executes until condition becomes false.

Recursive CTE Use Cases

Common real-world use cases:

- Employee hierarchy
 - Category hierarchy
 - Folder structure
 - Organization tree
 - Recursive reporting chains
-

CTE vs Subquery

Feature	Subquery	CTE
Readability	Lower	Higher
Reusability	Difficult	Easy
Query Structure	Nested	Step-based
Debugging	Harder	Easier
Maintainability	Lower	Better

When to Use CTE

Use CTE when:

- Query is becoming complex
- Multiple aggregations are involved
- Same logic is reused
- Query readability matters
- Step-by-step transformation is required
- Reporting queries are written

When Not to Use CTE

Avoid CTE for:

- Very simple queries
 - One-line filtering conditions
 - Unnecessary abstraction
-

Backend Developer Perspective

CTEs are useful in backend development for:

- Complex reporting APIs
- Analytics endpoints
- Pagination logic
- Hierarchical data retrieval
- Data transformation pipelines
- Reducing unreadable nested queries

Common usage areas:

- Spring Boot JPA native queries
 - Stored procedures
 - SQL optimization
-

Practice Questions

1. Find employees earning below average salary.
2. Find department-wise maximum salary.
3. Find employees earning above HR average salary.
4. Compare Finance average salary with overall average salary.
5. Find departments having more than one employee using CTE.
6. Create a recursive CTE generating numbers from 1 to 10.

Key Takeaways

- CTE is a temporary named result set.
- CTE improves query readability.
- CTE helps organize complex SQL logic.
- CTE is commonly used in reporting and analytics queries.
- Recursive CTE is useful for hierarchical data.
- CTE is preferred over deeply nested subqueries for maintainability.